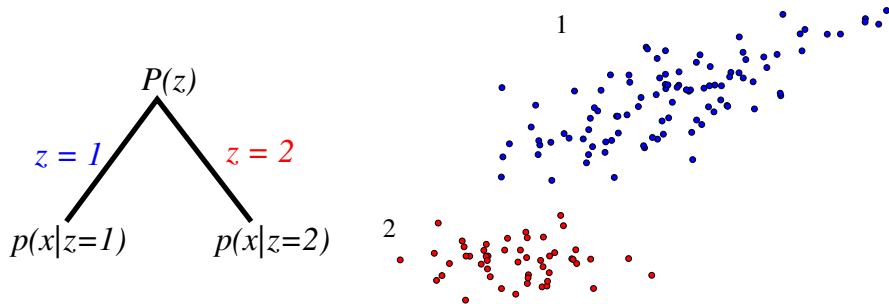


Machine Learning

Volker Roth

Department of Mathematics & Computer Science
University of Basel

Chapter 9: Mixture Models



Structure and mixtures

- Assume that input examples come in different potentially **unobserved types (groups, clusters, etc.)**.
- Assume that
 - 1 there are m underlying types $z = 1, \dots, m$;
 - 2 each type z occurs with probability $P(z)$;
 - 3 examples of type z distributed according to $p(\mathbf{x}|z)$.
- According to this model, each observed $\mathbf{x}$ comes from a **mixture distribution**:

$$p(\mathbf{x}) = \sum_{j=1}^m \underbrace{P(z=j)}_{\pi_j} p(\mathbf{x}|z=j, \theta_j)$$

- In many practical data analysis problems (such as probabilistic clustering), we want to **estimate** such parametric models from **samples** $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$. In particular, we are often interested in finding the types that have generated the examples.

Mixture of Gaussians

A mixture of Gaussians model has the form

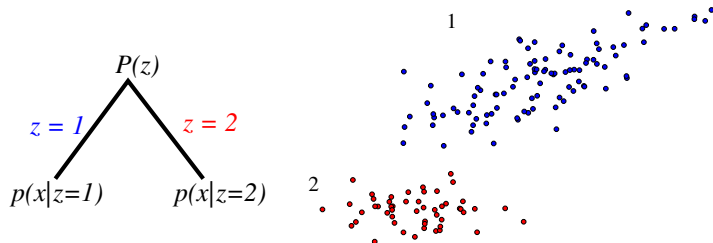
$$p(\mathbf{x}|\boldsymbol{\theta}) = \sum_{j=1}^m \pi_j \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j),$$

where $\boldsymbol{\theta} = \pi_1, \dots, \pi_m, \boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_m, \boldsymbol{\Sigma}_1, \dots, \boldsymbol{\Sigma}_m$ contains all the parameters. $\{\pi_j\}$ are the **mixing proportions**.



Mixture densities

- Data generation process:

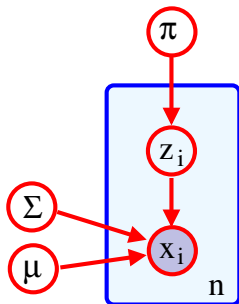


$$p(\mathbf{x}|\boldsymbol{\theta}) = \sum_{j=1}^m \pi_j p(\mathbf{x}|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$$

- Any data point $\mathbf{x}$ could have been **generated in two ways**.
 $\rightsquigarrow$ the responsible component needs to be **inferred**.

Mixtures as Latent Variable Models

- In the model $p(\mathbf{x}|z = j, \boldsymbol{\theta})$ the class indicator variable z is **latent**. This is an example of a large class of **latent variable models** (LVM).
- Bayesian network (DAG) = graphical representation of the joint distribution of RVs (nodes) as $P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{parents}(x_i))$



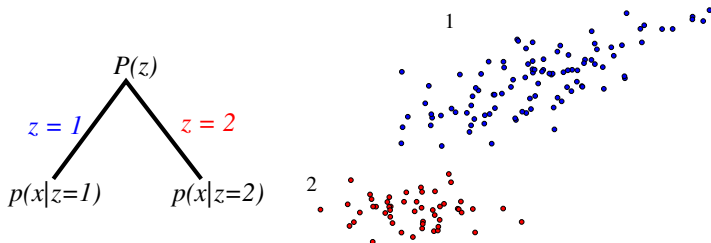
$$\begin{aligned} p(\mathbf{x}_i | \boldsymbol{\theta}) &= \sum_{z_i} p(\mathbf{x}_i, z_i | \boldsymbol{\theta}) \\ &= \sum_{z_i} p(\mathbf{x}_i | \boldsymbol{\mu}, \boldsymbol{\Sigma}, z_i) p(z_i | \pi). \end{aligned}$$

Mixture densities

- Consider a two component mixture of Gaussians model.

$$p(\mathbf{x}|\boldsymbol{\theta}) = \pi_1 p(\mathbf{x}|\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1) + \pi_2 p(\mathbf{x}|\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)$$

- If we knew the generating component $z_i = \{1, 2\}$ for each example $\mathbf{x}_i$, then the estimation would be easy.



- In particular, we can estimate each Gaussian independently.

Mixture density estimation

- Let $\delta(j|i)$ be an indicator function of whether example i is labeled j .
Then for each $j = 1, 2$

$$\hat{\pi}_j \leftarrow \frac{\hat{n}_j}{n}, \quad \text{where } \hat{n}_j = \sum_{i=1}^n \delta(j|i)$$

$$\hat{\mu}_j \leftarrow \frac{1}{\hat{n}_j} \sum_{i=1}^n \delta(j|i) \mathbf{x}_i$$

$$\hat{\Sigma}_j \leftarrow \frac{1}{\hat{n}_j} \sum_{i=1}^n \delta(j|i) (\mathbf{x}_i - \mu_j)(\mathbf{x}_i - \mu_j)^t$$

Mixture density estimation

- We don't have such labels... but we can guess what the labels might be based on our current distribution.
- One possible choice: evaluate posterior probability that an observed $\mathbf{x}$ was generated from first component

$$\begin{aligned} P(z = 1|\mathbf{x}, \boldsymbol{\theta}) &= \frac{P(z = 1) \cdot p(\mathbf{x}|z = 1)}{\sum_{j=1,2} P(z = j) \cdot p(\mathbf{x}|z = j)} \\ &= \frac{\pi_1 p(\mathbf{x}|\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1)}{\sum_{j=1,2} \pi_j p(\mathbf{x}|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} \end{aligned}$$

↪ Information about the component responsible for generating $\mathbf{x}$.

- **Soft labels** or **posterior probabilities**

$$\hat{p}(j|i) \leftarrow P(z_i = j|\mathbf{x}_i, \boldsymbol{\theta}),$$

where $\sum_{j=1,2} \hat{p}(j|i) = 1, \forall i = 1, \dots, n$.

The EM algorithm: iteration k

- **E-step:** softly assign examples to mixture components

$$\hat{p}(j|i) \leftarrow P(z_i = j | \mathbf{x}_i, \boldsymbol{\theta}^t), \forall j = 1, 2 \text{ and } i = 1, \dots, n.$$

Note: superscript is time index.

- **M-step:** estimate new mixture parameters $\boldsymbol{\theta}^{t+1}$ based on the soft assignments (can be done separately for the two Gaussians)

$$\hat{\pi}_j \leftarrow \frac{\hat{n}_j}{n}, \quad \text{where } \hat{n}_j = \sum_{i=1}^n \hat{p}(j|i)$$

$$\hat{\boldsymbol{\mu}}_j \leftarrow \frac{1}{\hat{n}_j} \sum_{i=1}^n \hat{p}(j|i) \mathbf{x}_i$$

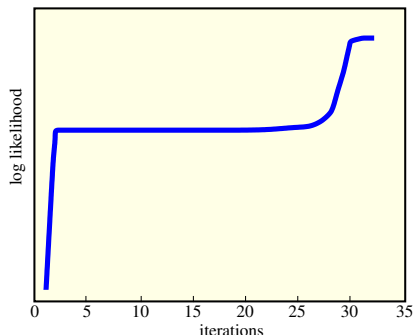
$$\hat{\boldsymbol{\Sigma}}_j \leftarrow \frac{1}{\hat{n}_j} \sum_{i=1}^n \hat{p}(j|i) (\mathbf{x}_i - \boldsymbol{\mu}_j)(\mathbf{x}_i - \boldsymbol{\mu}_j)^t$$

The EM-algorithm: Convergence

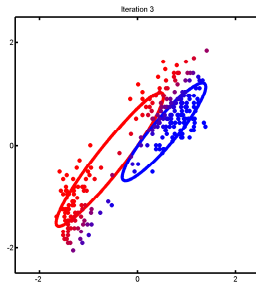
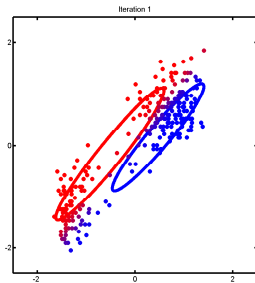
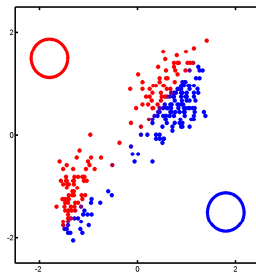
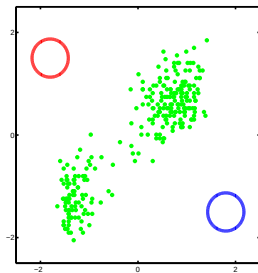
The EM-algorithm **monotonically increases the log-likelihood** of the training data (we will show this later). In other words,

$$l(\theta^0) < l(\theta^1) < l(\theta^2) < \dots \text{ until convergence}$$

$$l(\theta^t) = \sum_{i=1}^n \log p(\mathbf{x}_i | \theta^t).$$



Mixture density estimation: example



Mixture density estimation: example

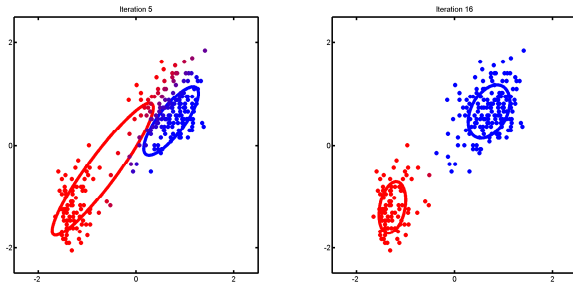
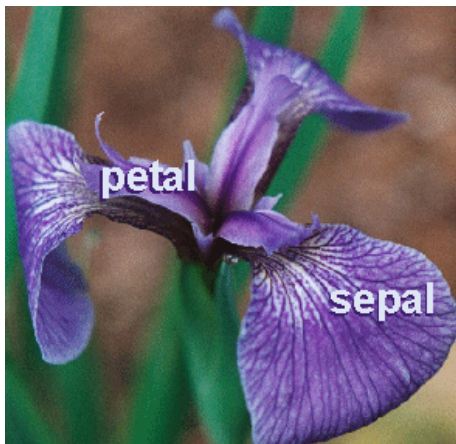
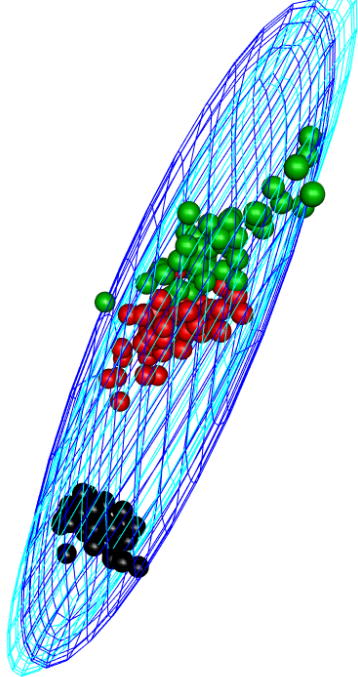


Fig. 11.11 in K. Murphy: Machine Learning. MIT Press 2012.

EM example: Iris data

- The famous (Fisher's or Anderson's) iris data set gives the measurements in centimeters of the variables **sepal length and width** and **petal length and width**, respectively, for 50 flowers from each of **3 species of iris**.
- The species are **Iris setosa, versicolor, and virginica**.





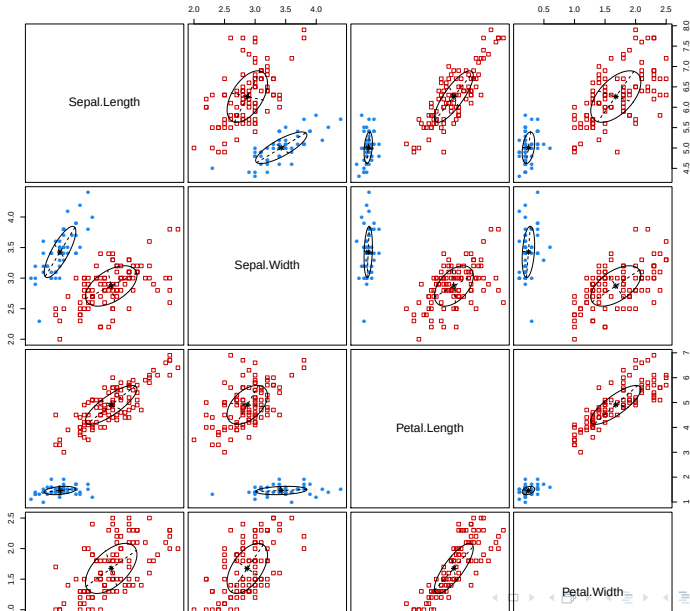
Bayesian model selection for mixture models

As a simple strategy for selecting the appropriate number of mixture components, we can find m that minimizes the overall description length (cf. BIC):

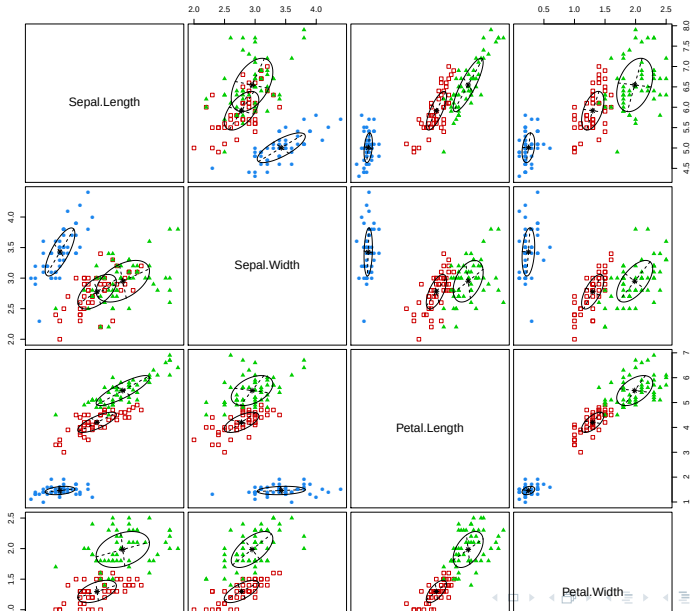
$$DL \approx -\log p(\text{data}|\hat{\theta}_m) + \frac{d_m}{n} \log(n)$$

- n is the number of training points,
- $\hat{\theta}_m$ are the maximum likelihood parameters for the m -component mixture, and
- d_m is the number of parameters in the m -component mixture.

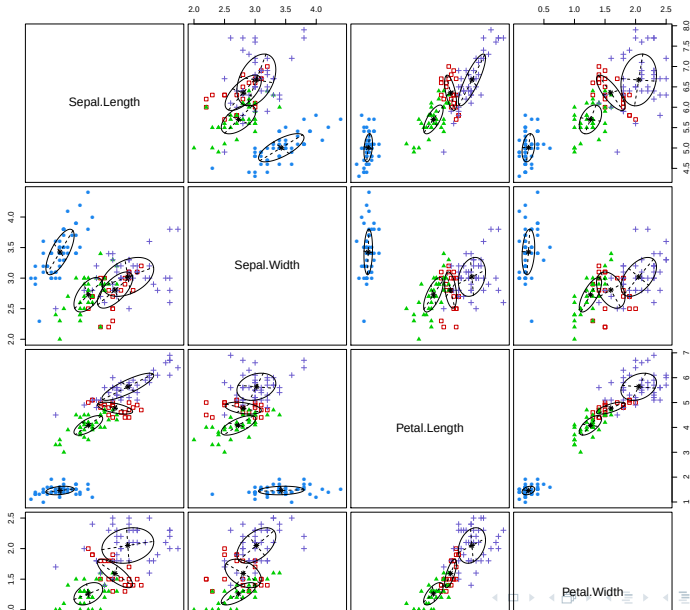
Model selection example: Iris data, $m = 2$



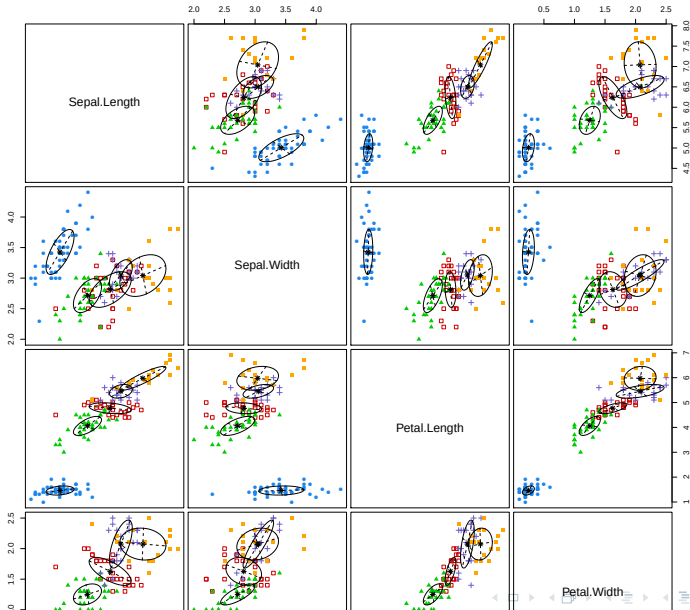
Model selection example: Iris data, $m = 3$



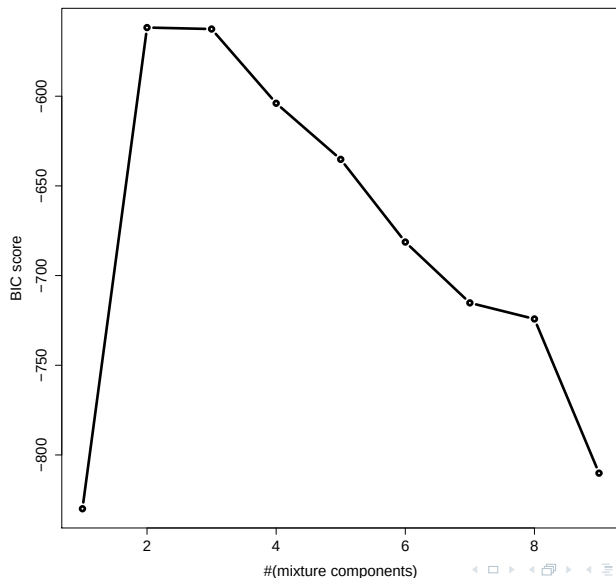
Model selection example: Iris data, $m = 4$



Model selection example: Iris data, $m = 5$



Model selection example: Iris data, BIC



The EM-algorithm: Convergence

Step 0: specify the initial setting of the parameters $\theta = \theta^0$.

E-step: complete the incomplete data (missing z) with the posterior probabilities (“soft labels”)

$$P(z = j | \mathbf{x}_i, \theta^t), \quad j = 1, \dots, m, \quad i = 1, \dots, n.$$

M-step: find the new setting of the parameters θ^{t+1} by maximizing the log-likelihood of the inferred (or “expected complete”) data

$$\theta^{t+1} = \arg \max_{\theta} \underbrace{\sum_{i=1}^n \sum_{j=1}^m P(z = j | \mathbf{x}_i, \theta^t) \log[p_j p(\mathbf{x}_i | \theta_j)]}_{\text{inferred (= expected complete) log-likelihood } Q(\theta, \theta^t)}.$$

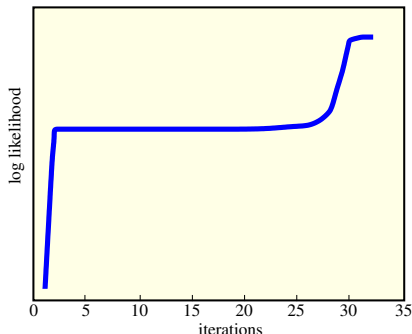
The expected complete log-likelihood $Q(\theta, \theta^t)$ is called **auxiliary objective**.

The EM-algorithm: Convergence

The EM-algorithm **monotonically increases the log-likelihood** of the training data. In other words,

$$l(\theta^0) < l(\theta^1) < l(\theta^2) < \dots \text{ until convergence}$$

$$l(\theta^t) = \sum_{i=1}^n \log p(\mathbf{x}_i | \theta^t).$$



Jensen's inequality

- Convex function: secant line above graph of the function
 $\rightsquigarrow$ Jensen's inequality for two points.
- Secant line consists of weighted means of the convex function.
For $a \in [0, 1]$:

$$af(x_1) + (1 - a)f(x_2).$$

Graph: convex function of the weighted means:

$$f(ax_1 + (1 - a)x_2).$$

- Thus, Jensen's inequality is

$$f(ax_1 + (1 - a)x_2) \leq af(x_1) + (1 - a)f(x_2).$$

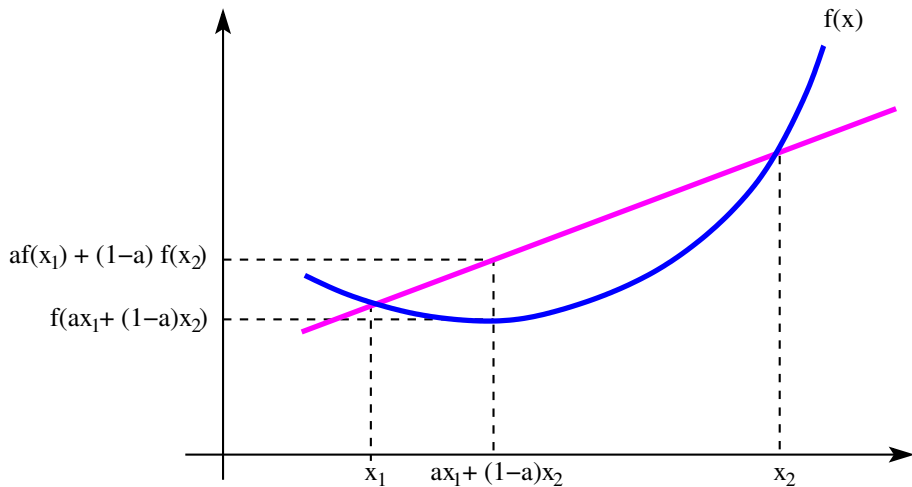
- Probability theory: if X is a RV and φ a convex function, then

$$\varphi(E[X]) \leq E[\varphi(X)].$$

- φ convex $\rightsquigarrow \psi := -\varphi$ concave:

$$\psi(E[X]) \geq E[\psi(X)]. \quad \text{Example:} \quad \log(E[X]) \geq E[\log(X)].$$

Jensen's inequality



Non-negativity of KL divergence

$$\begin{aligned} -\mathbb{KL}(p(x) \| q(x)) &= \int p(x) \log \left(\frac{q(x)}{p(x)} \right) dx \\ (\text{Jensen's inequality}) \quad &\leq \log \left(\int p(x) \frac{q(x)}{p(x)} dx \right) \\ &= \log \left(\int q(x) dx \right) \\ &= \log(1) = 0 \end{aligned}$$

This is also called **Gibbs' inequality**.

The EM-algorithm: Theoretical basis

Consider distribution $q(z_i)$ over latent assignment variables.

Log-likelihood:

$$\begin{aligned}l(\boldsymbol{\theta}) &= \sum_{i=1}^n \log p(\mathbf{x}_i | \boldsymbol{\theta}) \\&= \sum_{i=1}^n \log \sum_{z_i} p(\mathbf{x}_i, z_i | \boldsymbol{\theta}) \\&= \sum_{i=1}^n \log \sum_{z_i} q(z_i) \frac{p(\mathbf{x}_i, z_i | \boldsymbol{\theta})}{q(z_i)} \\&= \sum_{i=1}^n \log \mathbb{E}_{q_i} \frac{p(\mathbf{x}_i, z_i | \boldsymbol{\theta})}{q(z_i)} \\(\text{Jensen's inequality}) \quad &\geq \sum_{i=1}^n \sum_{z_i} \textcolor{red}{q(z_i)} \log \frac{p(\mathbf{x}_i, z_i | \textcolor{blue}{\boldsymbol{\theta}})}{\textcolor{red}{q(z_i)}} \\&=: Q(\textcolor{blue}{\boldsymbol{\theta}}, \textcolor{red}{q}).\end{aligned}$$

- Lower bound

$$l(\theta) \geq Q(\theta, q) := \sum_{i=1}^n \sum_{z_i} q(z_i) \log \frac{p(\mathbf{x}_i, z_i | \theta)}{q(z_i)}$$

valid for any positive distribution q . Which one should we choose?

- Intuition: pick the q that yields the tightest lower bound.

This will be the **E-step**.

- At time t , assume we have chosen q^t based on current parameters θ^t .

In the next **M-step** we maximize the

expected complete log-likelihood:

$$\theta^{t+1} = \arg \max_{\theta} Q(\theta, \theta^t) = \arg \max_{\theta} \sum_{i=1}^n \mathbb{E}_{q_i^t} \log p(\mathbf{x}_i, z_i | \theta)$$

- Last equation follows from

$$Q(\theta, q) = \underbrace{\sum_{i=1}^n \mathbb{E}_{q_i} \log p(\mathbf{x}_i, z_i | \theta)}_{\text{Expected complete log-}l} + \sum_{i=1}^n \left[\underbrace{- \sum_{z_i} q(z_i) \log q(z_i)}_{h(q_i), \text{ independent of } \theta} \right].$$

The E-step

Re-write lower bound as

$$Q(\boldsymbol{\theta}, q) = \sum_i L(\boldsymbol{\theta}, q_i),$$

with

$$\begin{aligned} L(\boldsymbol{\theta}, q_i) &= \sum_{z_i} q(z_i) \log \frac{p(\mathbf{x}_i, z_i | \boldsymbol{\theta})}{q(z_i)} \\ &= \sum_{z_i} q(z_i) \log \frac{p(z_i | \mathbf{x}_i, \boldsymbol{\theta}) p(\mathbf{x}_i | \boldsymbol{\theta})}{q(z_i)} \\ &= \sum_{z_i} q(z_i) \log \frac{p(z_i | \mathbf{x}_i, \boldsymbol{\theta})}{q(z_i)} + \sum_{z_i} q(z_i) \log p(\mathbf{x}_i | \boldsymbol{\theta}) \\ &= \underbrace{-\text{KL}(q(z_i) \| p(z_i | \mathbf{x}_i, \boldsymbol{\theta}))}_{\text{always } \geq 0, \text{ and } = 0, \text{ if } q=p} + \underbrace{\log p(\mathbf{x}_i | \boldsymbol{\theta})}_{\text{independent of } q_i}. \end{aligned}$$

The E step

For $q_i^t(z_i) = p(z_i|\mathbf{x}_i, \boldsymbol{\theta}^t)$, the KL divergence is zero, and $L(\boldsymbol{\theta}^t, q_i)$ is maximized over all possible distributions q_i :

$$q_i^t(z_i) = p(z_i|\mathbf{x}_i, \boldsymbol{\theta}^t) = \arg \max_{q_i} L(\boldsymbol{\theta}^t, q_i) \quad (\rightsquigarrow \text{E-step})$$

$$L(\boldsymbol{\theta}^t, q_i^t) = \log p(\mathbf{x}_i|\boldsymbol{\theta}^t)$$

$$Q(\boldsymbol{\theta}^t, \boldsymbol{\theta}^t) = \sum_i \log p(\mathbf{x}_i|\boldsymbol{\theta}^t) = l(\boldsymbol{\theta}^t)$$

$\rightsquigarrow$ lower bound “touches” the log-likelihood

$\rightsquigarrow$ after the E-step, the auxiliary objective equals the log-likelihood

$\rightsquigarrow$ lower bound is tight after the E-step.

The E step

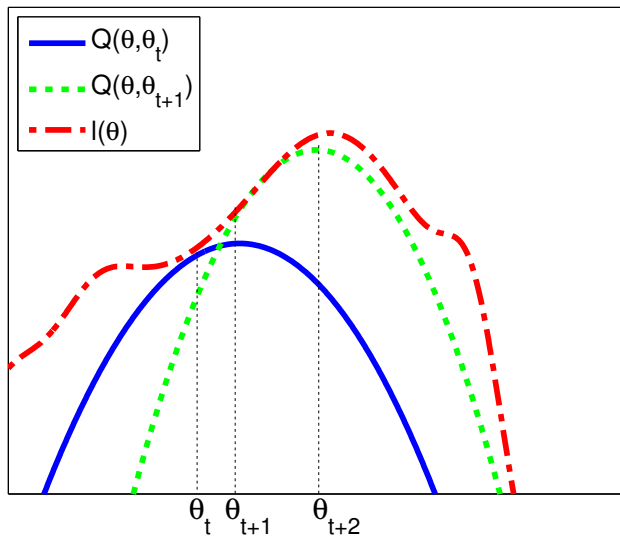


Fig 11.16 in K. Murphy: Machine Learning. MIT Press 2012.

EM-algorithm: max-max and monotonicity

We can now rewrite the EM-algorithm in terms of two maximization steps involving the auxiliary objective:

E-step: $\mathbf{q}^t = \arg \max_{\mathbf{q}} Q(\boldsymbol{\theta}^t, \mathbf{q})$

M-step: $\boldsymbol{\theta}^{t+1} = \arg \max_{\boldsymbol{\theta}} Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t)$.

The monotonic increase of the log-likelihood now follows from

$$\underbrace{l(\boldsymbol{\theta}^{t+1})}_{Q(\boldsymbol{\theta}^{t+1}, \bullet) \text{ is lower bound on } l(\boldsymbol{\theta}^{t+1})} \geq Q(\boldsymbol{\theta}^{t+1}, \boldsymbol{\theta}^t) \geq Q(\boldsymbol{\theta}^t, \boldsymbol{\theta}^t) = l(\boldsymbol{\theta}^t).$$

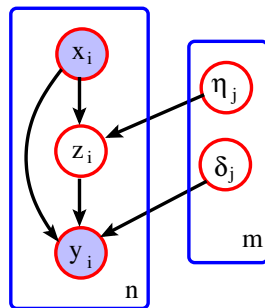
Second inequality: $Q(\boldsymbol{\theta}^{t+1}, \boldsymbol{\theta}^t) = \max_{\boldsymbol{\theta}} Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t) \geq Q(\boldsymbol{\theta}^t, \boldsymbol{\theta}^t)$.

Conditional mixtures

- Some regression or classification problems can be decomposed into easier sub-problems.
- Examples:
 - ▶ style in handwritten character recognition
 - ▶ dialect/accent in speech recognition, etc.
- Each sub-problem could be solved by a specific “expert”.
- The selection of which expert to rely on now depends on the position x in the input space. **Mixtures of experts models.**

Experts (regression)

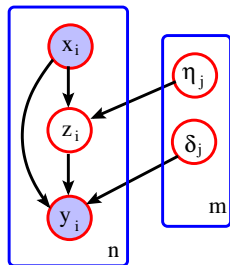
- Suppose we have several regression “experts” generating conditional Gaussian outputs
$$p(y|\mathbf{x}, z = j, \boldsymbol{\theta}) = \mathcal{N}(y|\boldsymbol{\beta}_j^t \mathbf{x}, \sigma_j^2)$$
- $\boldsymbol{\delta}_j = \{\boldsymbol{\beta}_j, \sigma_j^2\}$: Parameters of j -th expert.
- Need to find a way of allocating tasks to these experts.
- Parameter vector $\boldsymbol{\theta}$ contains the means and variances of the m experts and the additional parameters $\boldsymbol{\eta}$ of this allocation mechanism:
$$\boldsymbol{\theta} = \{\boldsymbol{\delta}_j, \boldsymbol{\eta}_j\}_{j=1}^m.$$



Joint distribution

From the DAG we conclude:

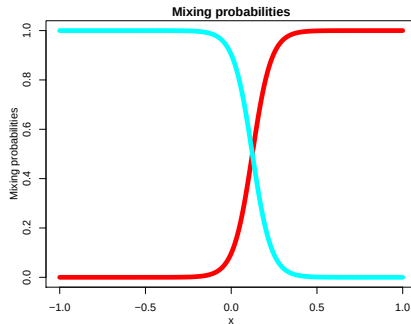
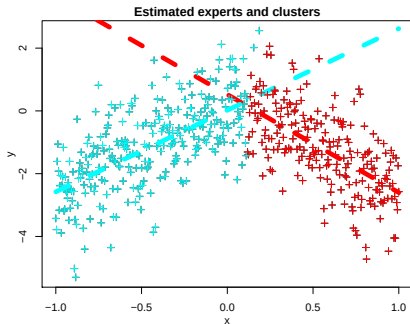
$$\begin{aligned} p(y, z = j | \mathbf{x}) &= p(y | \boldsymbol{\delta}, z = j, \mathbf{x}) P(z = j | \boldsymbol{\eta}, \mathbf{x}) \\ &= p(y | \boldsymbol{\delta}_j, \mathbf{x}) P(z = j | \boldsymbol{\eta}, \mathbf{x}) \\ &= \mathcal{N}(y | \boldsymbol{\beta}_j^t \mathbf{x}, \sigma_j^2) P(z = j | \boldsymbol{\eta}, \mathbf{x}) \end{aligned}$$



Thus, the overall prediction is

$$\begin{aligned} p(y | \mathbf{x}, \boldsymbol{\theta}) &= \sum_j p(y, z = j | \mathbf{x}, \boldsymbol{\eta}, \boldsymbol{\delta}) \\ &= \sum_j P(z = j | \mathbf{x}, \boldsymbol{\eta}) p(y | \mathbf{x}, \boldsymbol{\delta}_j) \\ &= \sum_j P(z = j | \mathbf{x}, \boldsymbol{\eta}) p(y | \mathbf{x}, \boldsymbol{\beta}_j, \sigma_j^2). \end{aligned}$$

Mixtures of experts



Here we need to switch from one linear regression model to another:
 $p(y|\mathbf{x}, z = j, \boldsymbol{\theta}) = \mathcal{N}(y|\boldsymbol{\beta}_j^t \mathbf{x}, \sigma_j^2)$. The switch can be probabilistic
 $\rightsquigarrow$ probabilistic gating function $P(z|\mathbf{x}, \boldsymbol{\eta})$ (right).

Gating network

- A gating network specifies a distribution over m experts, conditionally on the input $\mathbf{x}$.
- Example: when there are just two experts the gating network can be a logistic regression model

$$P(z = 1|\mathbf{x}, \boldsymbol{\eta}) = \sigma(\boldsymbol{\eta}^t \mathbf{x}),$$

where $\sigma(z) = (1 + e^{-z})^{-1}$ is the logistic function.

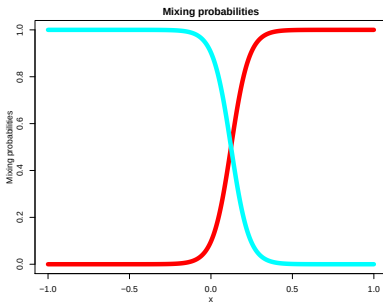
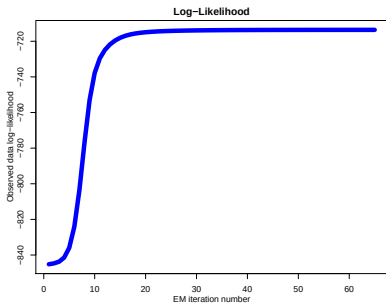
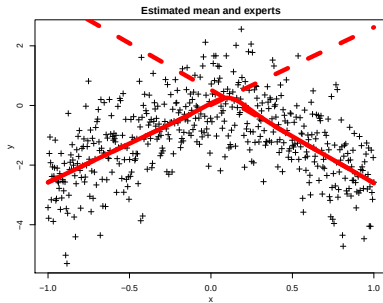
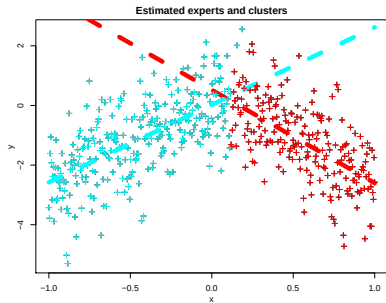
- For $m > 2$, the gating network can be a softmax model

$$P(z = j|\mathbf{x}, \boldsymbol{\eta}) = \frac{\exp(\boldsymbol{\eta}_j^t \mathbf{x})}{\sum_{j'=1}^m \exp(\boldsymbol{\eta}_{j'}^t \mathbf{x})},$$

where $\boldsymbol{\eta} = \{\boldsymbol{\eta}_1, \dots, \boldsymbol{\eta}_m\}$ are the parameters of the gating network.

- Overall prediction

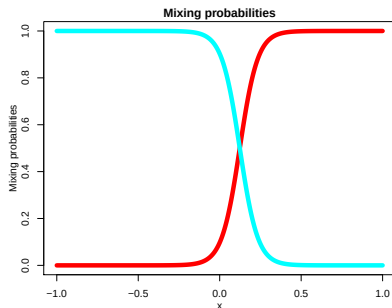
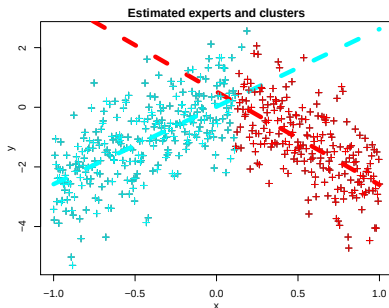
$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \sum_j p(y, z = j|\mathbf{x}, \boldsymbol{\eta}, \boldsymbol{\delta}) = \sum_j P(z = j|\mathbf{x}, \boldsymbol{\eta})p(y|\mathbf{x}, \boldsymbol{\delta}_j).$$



A mixture of experts model: estimation

“Soft labels”: Conditional probability that $(\mathbf{x}_i, y_i)$ came from expert j :

$$\begin{aligned}\hat{P}(j|i) &= P(z = j | \mathbf{x}_i, y_i, \boldsymbol{\theta}) \\ &= \frac{P(z = j | \mathbf{x}_i, \boldsymbol{\eta}^t) p(y_i | \mathbf{x}_i, (\beta_j, \sigma_j^2))}{\sum_{j'=1}^m P(z = j' | \mathbf{x}_i, \boldsymbol{\eta}^t) p(y_i | \mathbf{x}_i, (\beta_{j'}, \sigma_{j'}^2))}\end{aligned}$$



EM for mixtures of experts

E-step: compute soft labels $\hat{P}(j|i)$

M-step: separately re-estimate the experts and the gating network based on these soft assignments:

- 1 For each expert j : find $(\hat{\beta}_j, \hat{\sigma}_j^2)$ that maximize

$$\sum_{i=1}^n \hat{P}(j|i) \log p(y_i | \mathbf{x}_i, (\beta_j, \sigma_j^2))$$

$\rightsquigarrow$ linear regression with weighted training set.

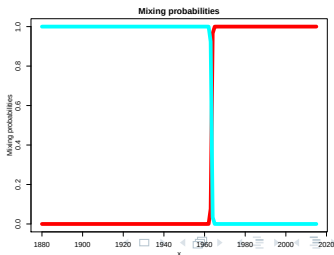
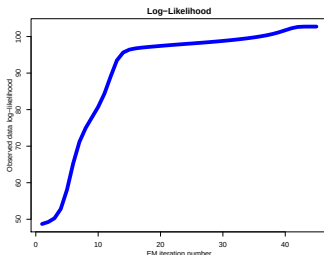
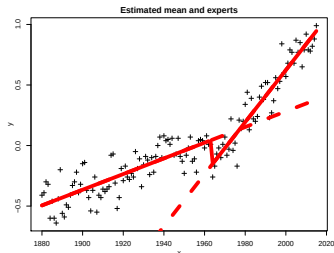
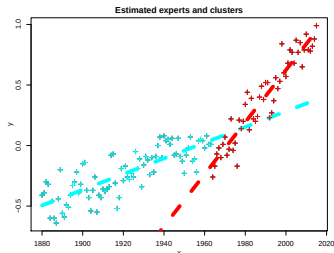
- 2 For the gating network: find $\hat{\eta}$ that maximize

$$\sum_{i=1}^n \sum_{j=1}^m \hat{P}(j|i) \log P(j | \mathbf{x}_i, \eta_j)$$

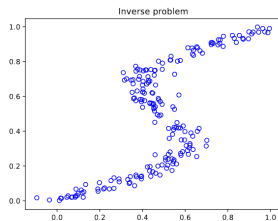
$\rightsquigarrow$ logistic regression with weighted training set.

Real World Example

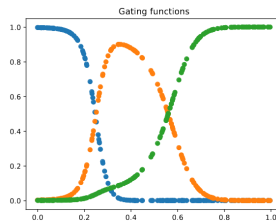
Global annual temperature anomalies (degrees C) computed using data from land meteorological stations, 1880-2015. Anomalies are relative to the 1951-1980 base period means.



Conditional Mixtures for Inverse Problems



(a)



(b)

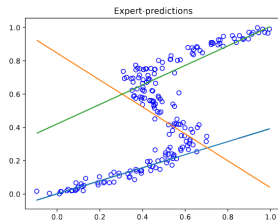
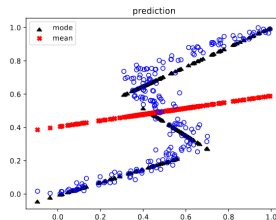


Figure 13.23 in K.Murphy: Probabilistic Machine Learning. MIT Press, 2023. a) data one-to-many function. (b) Responsibilities of each expert. (c) Prediction of each expert. (d) Overall prediction. Mean is red, mode is black.



Chapter 10: Linear latent variable models

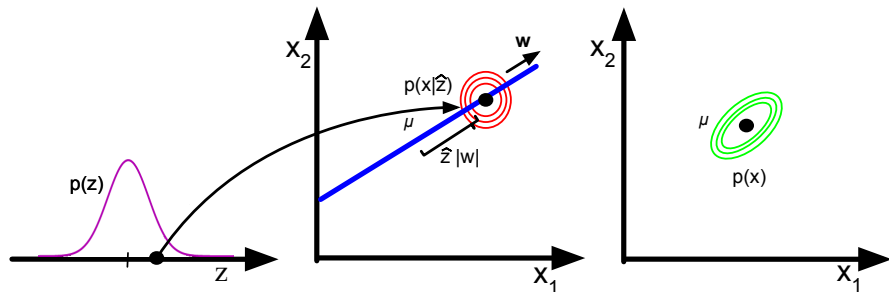
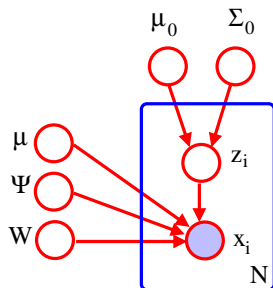


Figure 12.1 in K. Murphy: Machine Learning. MIT Press 2012.

Factor analysis

- One problem with mixture models: **only a single latent variable**. Each observation can only come from one of K prototypes.
- Alternative: $\mathbf{z}_i \in \mathbb{R}^k$. Gaussian prior:

$$p(\mathbf{z}_i) = \mathcal{N}(\mathbf{z}_i | \boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0)$$



- For observations $\mathbf{x}_i \in \mathbb{R}^p$, we may use a **Gaussian likelihood**.
- As in linear regression, we assume the mean is a **linear** function:

$$p(\mathbf{x}_i | \mathbf{z}_i, \boldsymbol{\theta}) = \mathcal{N}(W\mathbf{z}_i + \boldsymbol{\mu}, \Psi),$$

W : **factor loading matrix**, and Ψ : **covariance matrix**.

- We take Ψ to be **diagonal**, since the whole point of the model is to “force” $\mathbf{z}_i$ to **explain the correlation**.

Factor analysis: generative process

Generative process ($k = 1$, $p = 2$, diagonal Ψ):

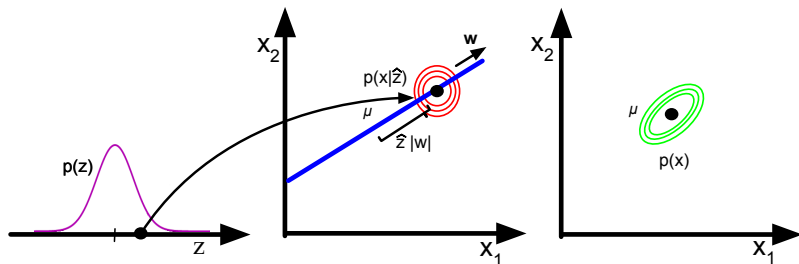


Figure 12.1 in K. Murphy: Machine Learning. MIT Press 2012.

We take an isotropic Gaussian “spray can” and slide it along the 1d line defined by $wz_i + \mu$. This induces a correlated Gaussian in 2d.

FA is a low rank parameterization of an MVN

- The induced marginal distribution $p(\mathbf{x}_i|\boldsymbol{\theta})$ is Gaussian:

$$\begin{aligned} p(\mathbf{x}_i|\boldsymbol{\theta}) &= \int \mathcal{N}(\mathbf{x}_i, W\mathbf{z}_i + \boldsymbol{\mu}, \Psi) \mathcal{N}(\mathbf{z}_i|\boldsymbol{\mu}_0, \Sigma_0) d\mathbf{z}_i, \\ &= \mathcal{N}(\mathbf{x}_i|W\boldsymbol{\mu}_0 + \boldsymbol{\mu}, \Psi + W\Sigma_0W^t) \end{aligned}$$

- We can set $\boldsymbol{\mu}_0 = \mathbf{0}$ without loss of generality (absorb $W\boldsymbol{\mu}_0$ into $\boldsymbol{\mu}$). Similarly, we can set $\Sigma_0 = I$ using $\tilde{W} = W\Sigma_0^{-1/2}$.

Covariance structure:

$$\text{cov}[\mathbf{x}|\boldsymbol{\theta}] = (W\Sigma_0^{-1/2})\Sigma_0(W\Sigma_0^{-1/2})^t + \Psi = WW^t + \Psi.$$

- We thus see that FA approximates the covariance matrix of $\mathbf{x}$ using a low-rank decomposition:

$$C := \text{cov}[\mathbf{x}] = WW^t + \Psi.$$

Only $O(p \cdot k)$ parameters $\rightsquigarrow$ compromise between full covariance with $O(p^2)$ params, and a diagonal covariance $O(p)$ params.

Inference of the latent factors

- We hope that the latent factors $\mathbf{z}$ will reveal something interesting about the data $\rightsquigarrow$ compute posterior over the latent variables:

$$\begin{aligned}p(\mathbf{z}_i | \mathbf{x}_i, \boldsymbol{\theta}) &= \mathcal{N}(\mathbf{z}_i | \mathbf{m}_i, \Sigma) \\ \Sigma &= (I + W^t \Psi^{-1} W)^{-1} \\ \mathbf{m}_i &= \Sigma W^t \Psi^{-1} \mathbf{x}_i\end{aligned}$$

- The posterior means $\mathbf{m}_i$ are called the **latent scores, or latent factors.**

Example

- Example from (Shalizi 2009). $p = 11$ variables and $n = 387$ cases describing aspects of cars: engine size, $\#(\text{cylinders})$, miles per gallon (MPG), price, etc.
- Fit a $p = 2$ dim model. Plot m_i scores as points in $\mathbb{R}^2$.
- To get a better understanding of the “meaning” of the latent factors, project unit vectors $e_1 = (1, 0, \dots, 0)$, $e_2 = (0, 1, 0, \dots, 0)$, etc. into the low dimensional space (blue lines)
- Horizontal axis represents price, corresponding to the features labeled “dealer” and “retail”, with expensive cars on the right. Vertical axis represents fuel efficiency (measured in terms of MPG) versus size: heavy vehicles are less efficient and are higher up, whereas light vehicles are more efficient and are lower down.
- Verify by finding the closest exemplars in the training set.

Example

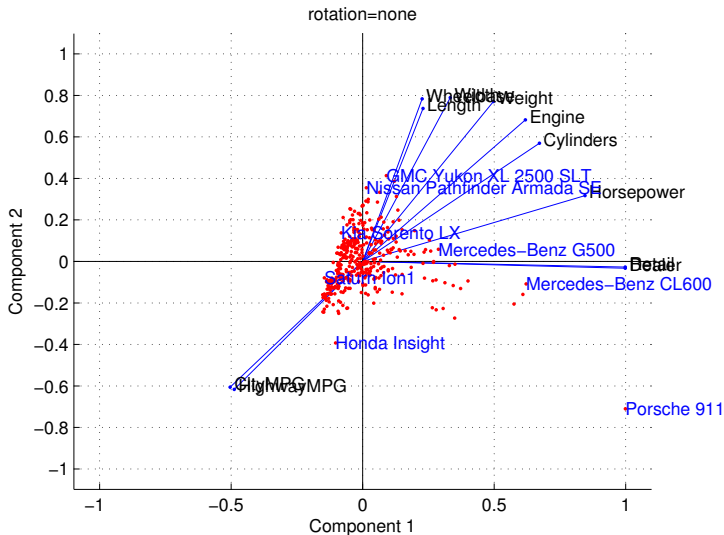
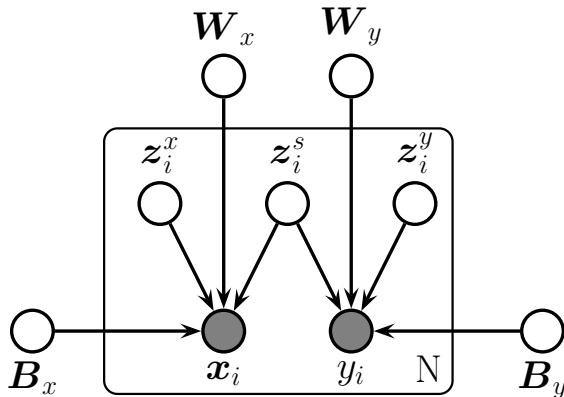


Figure 12.2 in K. Murphy: Machine Learning. MIT Press 2012.

Special Cases: PCA and CCA

- Covariance matrix $\Psi = \sigma^2 I \rightsquigarrow$ (probabilistic) **PCA**.
- Two-view version involving $\mathbf{x}$ and $\mathbf{y} \rightsquigarrow$ **CCA**.



From figure 12.19 in K. Murphy: Machine Learning. MIT Press 2012.

PCA and dimensionality reduction

Given n data points in p dimensions:

$$X = \begin{bmatrix} - & \mathbf{x}_1 & - \\ - & \mathbf{x}_2 & - \\ - & \vdots & - \\ - & \mathbf{x}_n & - \end{bmatrix} \in \mathbb{R}^{n \times p}$$

Want to reduce dimensionality from p to k . Choose k directions $\mathbf{w}_1, \dots, \mathbf{w}_k$, arrange them as columns in matrix W :

$$W = \begin{bmatrix} \mathbf{w}_1 & \mathbf{w}_2 & \dots & \mathbf{w}_k \end{bmatrix} \in \mathbb{R}^{p \times k}$$

For each $\mathbf{w}_j$, compute **similarity** $z_j = \mathbf{w}_j^t \mathbf{x}$, $j = 1 \dots k$.

Project $\mathbf{x}$ down to $\mathbf{z} = (z_1, \dots, z_k)^t = W^t \mathbf{x}$. How to choose W ?

Encoding-decoding model

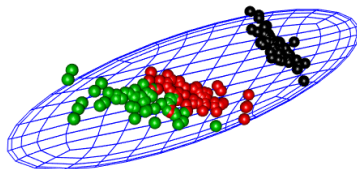
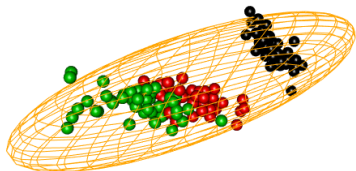
The projection matrix W serves two functions:

- **Encode:** $\mathbf{z} = W^t \mathbf{x}$, $\mathbf{z} \in \mathbb{R}^k$, $z_j = \mathbf{w}_j^t \mathbf{x}$.
 - ▶ The vectors $\mathbf{w}_j$ form a basis of the projected space.
 - ▶ We will require that this basis is orthonormal, i.e. $W^t W = I$.
- **Decode:** $\tilde{\mathbf{x}} = W \mathbf{z} = \sum_{j=1}^k z_j \mathbf{w}_j$, $\tilde{\mathbf{x}} \in \mathbb{R}^p$.
 - ▶ If $k = p$, the above orthonormality condition implies $W^t = W^{-1}$, and encoding can be undone without loss of information.
 - ▶ If $k < p$, we use the pseudo-inverse
 $\leadsto$ the reconstruction error will be nonzero.
- Above we assumed that the origin of the coordinate system is in the sample mean, i.e. $\sum_i \mathbf{x}_i = 0$.

Principal Component Analysis (PCA)

In the general case, we want the reconstruction error $\|\mathbf{x} - \tilde{\mathbf{x}}\|$ to be small.

Objective: minimize $\min_{W \in \mathbb{R}^{p \times k}: W^t W = I} \sum_{i=1}^n \|\mathbf{x}_i - WW^t \mathbf{x}_i\|^2$



Finding the principal components

Projection vectors are orthogonal $\rightsquigarrow$ can treat them separately:

$$\begin{aligned} \min_{\mathbf{w}: \|\mathbf{w}\|=1} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{w}\mathbf{w}^t \mathbf{x}_i\|^2 \\ \sum_i \|\mathbf{x}_i - \mathbf{w}\mathbf{w}^t \mathbf{x}_i\|^2 &= \sum_{i=1}^n [\mathbf{x}_i^t \mathbf{x}_i - 2\mathbf{x}_i^t \mathbf{w}\mathbf{w}^t \mathbf{x}_i + \underbrace{\mathbf{x}_i^t \mathbf{w}\mathbf{w}^t \mathbf{w}\mathbf{w}^t \mathbf{x}_i}_{=1}] \\ &= \sum_i [\mathbf{x}_i^t \mathbf{x}_i - \mathbf{x}_i^t \mathbf{w}\mathbf{w}^t \mathbf{x}_i] \\ &= \sum_i \mathbf{x}_i^t \mathbf{x}_i - \mathbf{w}^t \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^t \mathbf{w} \\ &= \underbrace{\sum_i \mathbf{x}_i^t \mathbf{x}_i}_{\text{const.}} - \mathbf{w}^t \mathbf{X}^t \mathbf{X} \mathbf{w}. \end{aligned}$$

Finding the principal components

- Want to maximize $\mathbf{w}^t X^t X \mathbf{w}$ under the constraint $\|\mathbf{w}\| = 1$
- Can also maximize the ratio $J(\mathbf{w}) = \frac{\mathbf{w}^t X^t X \mathbf{w}}{\mathbf{w}^t \mathbf{w}}$ and rescale $\hat{\mathbf{w}}$.
- Optimal projection u is the eigenvector of $X^t X$ with largest eigenvalue (J is a Rayleigh quotient).
- Note that we assumed that $\sum_i \mathbf{x}_i = 0$. Thus, the columns of X are assumed to sum to zero.
 - ↪ compute SVD of “centered” matrix X
 - ↪ column vectors in W are eigenvectors of $X^t X$
 - ↪ they are the principal components.

Eigen-faces [Turk and Pentland, 1991]

- p = number of pixels
- Each $\mathbf{x}_i \in \mathbb{R}^p$ is a face image
- x_{ji} = intensity of the j -th pixel in image i

$$\begin{pmatrix} \text{img}_1 & \dots & \text{img}_n \end{pmatrix} \approx W_{p \times k} \begin{bmatrix} \mathbf{z}_1 & \dots & \mathbf{z}_n \end{bmatrix}$$

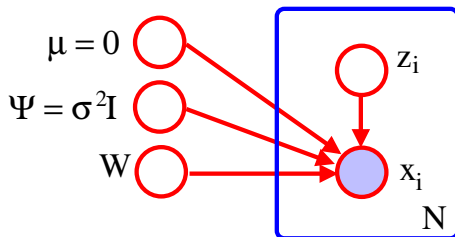
The equation shows a matrix of face images (represented by two example images and an ellipsis) is approximately equal to a weight matrix $W_{p \times k}$ multiplied by a matrix of coefficients $\mathbf{z}_1, \dots, \mathbf{z}_n$. The coefficient matrix is labeled $(Z^t)_{k \times n}$ at the top right.

Idea: $\mathbf{z}_i$ more 'meaningful' representation of i -th face than $\mathbf{x}_i$

Can use $\mathbf{z}_i$ for nearest-neighbor classification

Much faster when $p \gg k$.

Probabilistic PCA



- Assuming $\Psi = \sigma^2 I$ and centered data in the FA model
 $\rightsquigarrow$ likelihood and marginal likelihood

$$p(\mathbf{x}_i | \mathbf{z}_i, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{W} \mathbf{z}_i, \sigma^2 I),$$

$$\begin{aligned} p(\mathbf{x}_i | \boldsymbol{\theta}) &= \int \mathcal{N}(\mathbf{x}_i, \mathbf{W} \mathbf{z}_i, \sigma^2 I) \mathcal{N}(\mathbf{z}_i | \mathbf{0}, I) d\mathbf{z}_i, \\ &= \mathcal{N}(\mathbf{x}_i | \mathbf{0}, \sigma^2 I + \mathbf{W} \mathbf{W}^t) \end{aligned}$$

Probabilistic PCA

- (Tipping & Bishop 1999): Maxima of the marg. likelihood given by

$$\hat{W} = V(\Lambda - \sigma^2 I)^{\frac{1}{2}} R,$$

where R is an arbitrary orthogonal matrix,
columns of V : first k eigenvectors of $S = \frac{1}{n} X^t X$,
 Λ : diagonal matrix of eigenvalues.

- As $\sigma^2 \rightarrow 0$, we have $\hat{W} \rightarrow V$, **as in classical PCA** (for $R = \Lambda^{-\frac{1}{2}}$).
- Projections $\mathbf{z}_i$: Posterior over the latent factors:

$$p(\mathbf{z}_i | \mathbf{x}_i, \hat{\boldsymbol{\theta}}) = \mathcal{N}(\mathbf{z}_i | \hat{\mathbf{m}}_i, \sigma^2 \hat{\mathbf{F}}^{-1})$$

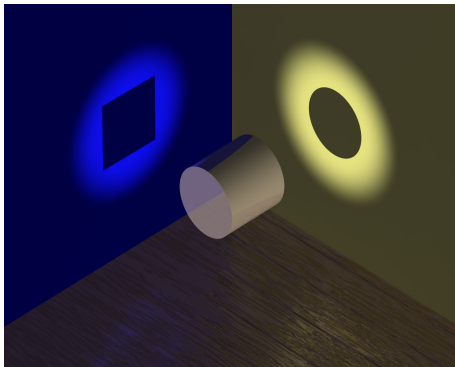
$$\hat{\mathbf{F}} = \sigma^2 I + \hat{W}^t \hat{W}$$

$$\mathbf{m}_i = \hat{\mathbf{F}}^{-1} \hat{W}^t \mathbf{x}_i$$

For $\sigma^2 \rightarrow 0$, $\mathbf{z}_i \rightarrow \mathbf{m}_i$ and $\mathbf{m}_i \rightarrow V^t \mathbf{x}_i \rightsquigarrow$ orthogonal projection of the data onto the column space of V , **as in classical PCA**.

Multiple Views: CCA

- Consider paired samples from different views.
- What is the dependency structure between the views ?
- Standard approach: [global linear](#) dependency detected by [CCA](#).



Canonical Correlation Analysis [Hotelling, 1936]

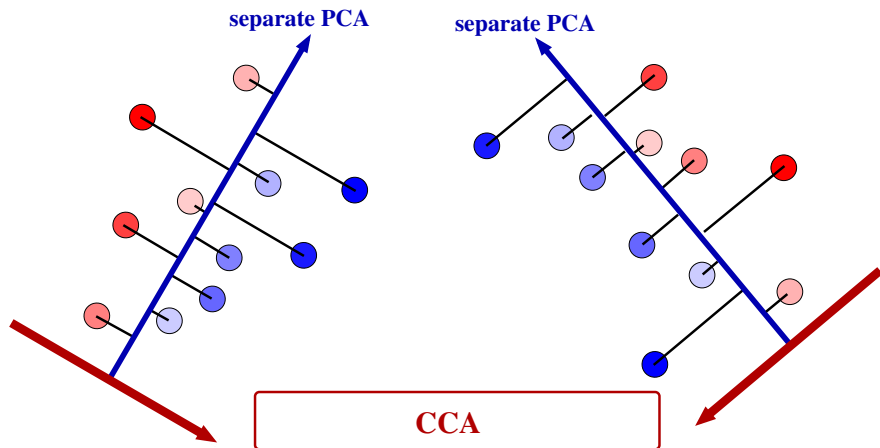
Often, each data point consists of **two views**:

- Image retrieval: for each image, have the following:
 - ▶ X : Pixels (or other visual features) Y : Text around the image
- Time series:
 - ▶ X : Signal at time t
 - ▶ Y : Signal at time $t + 1$
- Two-view learning: divide features into two sets
 - ▶ X : Features of a word/object, etc.
 - ▶ Y : Features of the context in which it appears

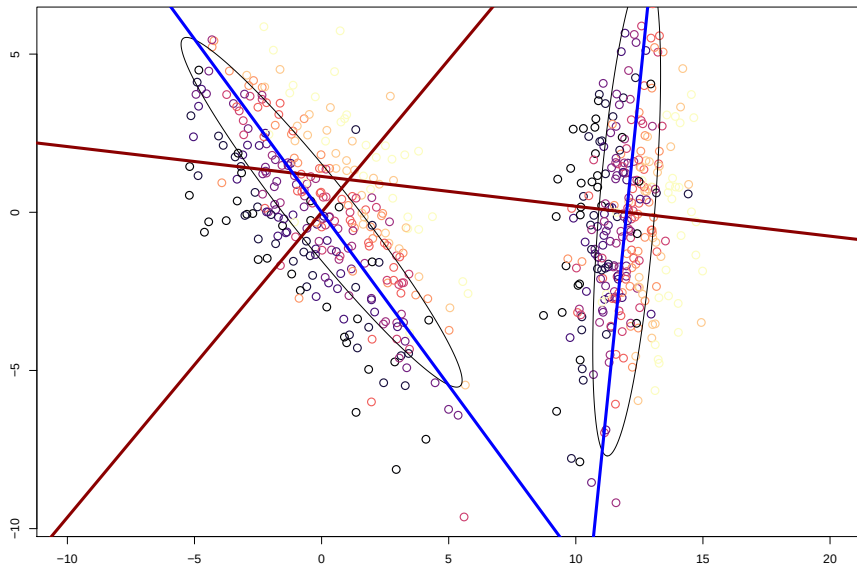
Goal: reduce the dimensionality of the two views **jointly**.

Find projections such that projected views are **maximally correlated**.

CCA vs PCA



CCA vs PCA



CCA: Setting

- Let $\mathbf{X}$ be a random vector $\in \mathbb{R}^{p_x}$ and $\mathbf{Y}$ be a random vector $\in \mathbb{R}^{p_y}$. Consider the combined $(p := p_x + p_y)$ -dimensional random vector $\mathbf{Z} = (\mathbf{X}, \mathbf{Y})^t$. Let its $(p \times p)$ covariance matrix be partitioned into blocks according to:

$$\mathbf{Z} = \left[\begin{array}{c|c} \Sigma_{XX} \in \mathbb{R}^{p_x \times p_x} & \Sigma_{XY} \in \mathbb{R}^{p_x \times p_y} \\ \hline \Sigma_{YX} \in \mathbb{R}^{p_y \times p_x} & \Sigma_{YY} \in \mathbb{R}^{p_y \times p_y} \end{array} \right]$$

- Assuming centered data, the blocks in the covariance matrix can be estimated from observed data sets $\mathbf{X} \in \mathbb{R}^{n \times p_x}$, $\mathbf{Y} \in \mathbb{R}^{n \times p_y}$:

$$\mathbf{Z} \approx \frac{1}{n} \left[\begin{array}{c|c} \mathbf{X}^t \mathbf{X} & \mathbf{X}^t \mathbf{Y} \\ \hline \mathbf{Y}^t \mathbf{X} & \mathbf{Y}^t \mathbf{Y} \end{array} \right]$$

CCA: Setting

- $\text{Correlation}(x, y) = \frac{\text{covariance}(x, y)}{\text{standard deviation}(x) \cdot \text{standard deviation}(y)}$

$$\rho = \text{cor}(x, y) = \frac{\text{cov}(x, y)}{\sigma(x)\sigma(y)}.$$

- Sample correlation:

$$\rho = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})^t}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}} \quad \text{centered observations} = \frac{\mathbf{x}^t \mathbf{y}}{\sqrt{\mathbf{x}^t \mathbf{x}} \sqrt{\mathbf{y}^t \mathbf{y}}}.$$

- Want to find maximally correlated 1D-projections $\mathbf{x}^t \mathbf{a}$ and $\mathbf{y}^t \mathbf{b}$.
- Projected covariance: $\text{cov}(\mathbf{x}^t \mathbf{a}, \mathbf{y}^t \mathbf{b}) \stackrel{\text{zero means}}{=} \mathbf{a}^t \Sigma_{XY} \mathbf{b}$.

- Define $\mathbf{c} = \Sigma_{XX}^{-\frac{1}{2}} \mathbf{a}$, $\mathbf{d} = \Sigma_{YY}^{-\frac{1}{2}} \mathbf{b}$.

- Thus, the projected correlation coefficient is: $\rho = \frac{\mathbf{c}^t \Sigma_{XX}^{-\frac{1}{2}} \Sigma_{XY} \Sigma_{YY}^{-\frac{1}{2}} \mathbf{d}}{\sqrt{\mathbf{c}^t \mathbf{c}} \sqrt{\mathbf{d}^t \mathbf{d}}}.$

CCA: Setting

- By the Cauchy-Schwarz inequality ($\mathbf{x}^t \mathbf{y} \leq \|\mathbf{x}\| \cdot \|\mathbf{y}\|$), we have

$$\left(\mathbf{c}^t \underbrace{\Sigma_{XX}^{-\frac{1}{2}} \Sigma_{XY} \Sigma_{YY}^{-\frac{1}{2}}}_H \mathbf{d} \right) \leq \left(\mathbf{c}^t \underbrace{\Sigma_{XX}^{-\frac{1}{2}} \Sigma_{XY} \Sigma_{YY}^{-1} \Sigma_{YX} \Sigma_{XX}^{-\frac{1}{2}}}_{G:=HH^t} \mathbf{c} \right)^{\frac{1}{2}} (\mathbf{d}^t \mathbf{d})^{\frac{1}{2}},$$
$$\rho \leq \frac{(\mathbf{c}^t G \mathbf{c})^{\frac{1}{2}}}{(\mathbf{c}^t \mathbf{c})^{\frac{1}{2}}},$$
$$\rho^2 \leq \frac{\mathbf{c}^t G \mathbf{c}}{\mathbf{c}^t \mathbf{c}}.$$

- Equality: vectors $\mathbf{d}$ and $\Sigma_{YY}^{-\frac{1}{2}} \Sigma_{YX} \Sigma_{XX}^{-\frac{1}{2}} \mathbf{c}$ are collinear.
- Maximum: $\mathbf{c}$ is the eigenvector with the maximum eigenvalue of $G := \Sigma_{XX}^{-\frac{1}{2}} \Sigma_{XY} \Sigma_{YY}^{-1} \Sigma_{YX} \Sigma_{XX}^{-\frac{1}{2}}$.
Subsequent pairs $\rightsquigarrow$ using eigenvalues of decreasing magnitudes.
- Collinearity: $\mathbf{d} \propto \Sigma_{YY}^{-\frac{1}{2}} \Sigma_{YX} \Sigma_{XX}^{-\frac{1}{2}} \mathbf{c}$
- Transform back to original variables $\mathbf{a} = \Sigma_{XX}^{-\frac{1}{2}} \mathbf{c}$, $\mathbf{b} = \Sigma_{YY}^{-\frac{1}{2}} \mathbf{d}$.

Pixels That Sound [Kidron, Schechner, Elad, 2005]

“People and animals fuse auditory and visual information to obtain robust perception. A particular benefit of such cross-modal analysis is the ability to localize visual events associated with sound sources. We aim to achieve this using computer-vision aided by a single microphone”.



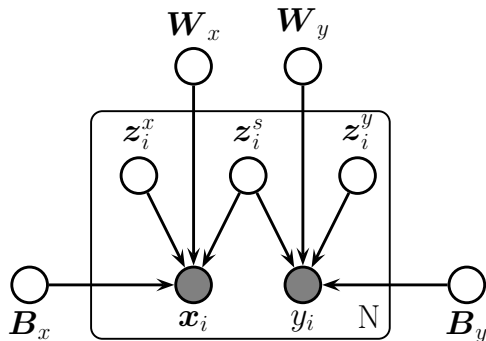
<https://webee.technion.ac.il/~yoav/research/pixels-that-sound.html>

Probabilistic CCA

(Bach and Jordan 2005): With Gaussian priors

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z}^s | \mathbf{0}, I) \mathcal{N}(\mathbf{z}^x | \mathbf{0}, I) \mathcal{N}(\mathbf{z}^y | \mathbf{0}, I),$$

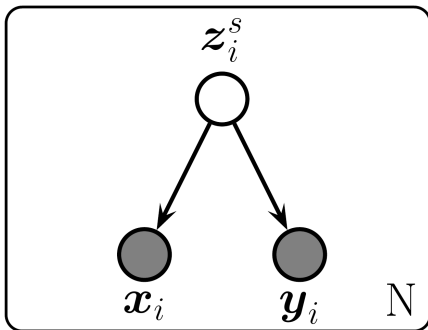
the MLE in the two-view FA model is equivalent to classical CCA (up to rotation and scaling).



From figure 12.19 in K. Murphy: Machine Learning. MIT Press 2012.

Probabilistic CCA: Essential Model Structure

Shared $\mathbf{z}^S$ decorrelates the two views $(\mathbf{x}, \mathbf{y})$:



$$\begin{aligned}\mathbf{x} &\in \mathbb{R}^p, \mathbf{y} \in \mathbb{R}^q \\ \mathbf{z}^S &\sim \mathcal{N}(\mathbf{z}^S | \mathbf{0}, I) \\ (\mathbf{x}, \mathbf{y}) | \mathbf{z} &\sim \mathcal{N}_{p+q}(\boldsymbol{\mu}_z, \boldsymbol{\Sigma}) \\ \boldsymbol{\Sigma} &= \begin{pmatrix} \boldsymbol{\Sigma}_x & 0 \\ 0 & \boldsymbol{\Sigma}_y \end{pmatrix}\end{aligned}$$

Further connections

- If y is a **discrete class label** $\rightsquigarrow$ CCA is (essentially) equivalent to Linear Discriminant Analysis (LDA), see (Hastie et al. 1994).
- Arbitrary y $\rightsquigarrow$ CCA is (essentially) equivalent to the **Gaussian Information Bottleneck** (Chechik et al. 2005)
 - ▶ Basic idea: **compress x** into compact latent representation while **preserving information about y** .
 - ▶ Information theoretic motivation:
Find encoding distribution $p(z|x)$ by minimizing
$$I(x; z) - \beta I(z; y)$$
where $\beta \geq 0$ is some parameter controlling the trade-off between compression and predictive accuracy.
- Arbitrary y , **discrete shared latent z^s**
 $\rightsquigarrow$ **dependency-seeking clustering** (Klami and Kaski 2008): find clusters that “explain” the dependency between the two views.